Optymalizacja globalna

Uniwersytet w Białymstoku kierunek Informatyka II stopnia, semestr 1

Sprawozdanie 1

Student: Dawid Szoka
Nr albumu: 89221
Rok akademicki: 2026/2027

Spis treści

1 Cel ćwiczenia **2**

2 Funkcja celu **2**

3 Algorytm krok po kroku **2**
3.1 Krok 1 – liczba możliwych wartości zmiennej 2
3.2 Krok 2 – wyznaczenie liczby bitów m 3
3.3 Krok 3 – losowanie populacji chromosomów 3
3.4 Krok 4 – podział chromosomu na zmienne 4
3.5 Krok 5 – zamiana na liczby dziesiętne 4
3.6 Krok 6 – skalowanie do przedziału $[a, b]$ 4
3.7 Krok 7 – dekodowanie populacji i wybór najlepszego 4

4 Testy dokładności **5**
4.1 Parametry kodowania 5
4.2 Wpływ liczebności populacji 6
4.3 Wpływ dokładności d 7
4.4 Czas obliczeń 7

5 Wizualizacja dla jednej zmiennej **9**

6 Wnioski **10**

1 Cel ćwiczenia

Celem ćwiczenia jest znalezienie minimum globalnego funkcji wielu zmiennych metodą losowego przeszukiwania z binarnym kodowaniem zmiennych. Każda zmienna x_i należy do przedziału $[a_i, b_i]$ z dokładnością d_i miejsc po przecinku, kodowana łańcuchem m_i bitów, losowana, dekodowana i skalowana z powrotem do przedziału. Spośród N wylosowanych punktów (populacji) wybierany jest ten o najmniejszej wartości funkcji celu.

W sprawozdaniu omówiono kolejne kroki algorytmu z pliku `lab2.py` wraz z kodem, a następnie zbadano, jak dokładność wyniku zależy od liczby zmiennych $n \in \{3, 5, 7, 9\}$, dokładności d oraz liczebności populacji N. Na koniec pokazano działanie metody dla jednej zmiennej ($n = 1$) na wykresie funkcji $f(x)$.

2 Funkcja celu

Minimalizowana jest n-wymiarowa funkcja typu Rastrigina:

$$f(x_1, \ldots, x_n) = 10n + \sum_{i=1}^{n} \left(x_i^2 - 10\cos(20\pi x_i)\right), \qquad x_i \in [-1, 1],$$

której minimum globalne to $f(0, \ldots, 0) = 0$.

```python
def func_nd(*args):
    n = len(args)

    suma = sum(x ** 2 - 10 * math.cos(20 * math.pi * x) for x in args)

    return 10 * n + suma
```

Funkcja przyjmuje dowolną liczbę argumentów (`*args`).

3 Algorytm krok po kroku

Całość spina funkcja `calc(data, n, f)`, gdzie `data` to lista słowników `{'a', 'b', 'd'}` (po jednym na zmienną), `n` to liczebność populacji N, a `f` to funkcja celu. Przykładowe wywołanie z pliku: 3 zmienne, $x_i \in [-1, 1]$, $d = 6$, $N = 1\,000\,000$.

```python
zmienne_3d = [
    {'a': -1, 'b': 1, 'd': 6}, # x1
    {'a': -1, 'b': 1, 'd': 6}, # x2
    {'a': -1, 'b': 1, 'd': 6}  # x3
]
najlepszy_punkt = calc(zmienne_3d, 1000000, func_nd)
```

3.1 Krok 1 – liczba możliwych wartości zmiennej

Zmienna z przedziału $[a, b]$ zapisana z dokładnością d miejsc po przecinku może przyjąć tylko L różnych wartości. Na przykład dla $[-1, 1]$ i $d = 1$ są to liczby $-1{,}0$; $-0{,}9$; …; $0{,}9$; $1{,}0$, czyli $L = 21$:

$$L_i = (b_i - a_i) \cdot 10^{d_i} + 1$$

```
length_a = [(x.get('b') - x.get('a')) * (10 ** x.get('d')) + 1 for x in
    data]
```

Dla $a = -1$, $b = 1$, $d = 6$: $L = 2 \cdot 10^6 + 1 = 2\,000\,001$ punktów na każdą zmienną.

3.2 Krok 2 – wyznaczenie liczby bitów m

Liczba bitów m to najmniejsza liczba, dla której 2^m pokrywa wszystkie L punktów:

$$2^{m_i - 1} < L_i \leq 2^{m_i} \quad \Longleftrightarrow \quad m_i = \lceil \log_2 L_i \rceil.$$

```
def find_m(population):
    result = 2
    m = 1
    while result < population:
        if result >= population:
            break
        result = result * 2
        m += 1
    return m

ms = [find_m(leng) for leng in length_a]
```

Pętla podwaja `result` ($= 2^m$), dopóki jest mniejszy od L. Dla $L = 2\,000\,001$: $2^{20} = 1\,048\,576 < L \leq 2^{21} = 2\,097\,152$, więc $m = 21$. Pętla wykonuje tylko m iteracji (ok. 20), więc zastąpienie jej wzorem z logarytmem nie daje zauważalnego zysku.

3.3 Krok 3 – losowanie populacji chromosomów

Każdy osobnik populacji to jeden ciąg binarny (chromosom) o długości

$$M = m_1 + m_2 + \cdots + m_n,$$

w którym kolejne odcinki kodują kolejne zmienne. Dla 3 zmiennych i $d = 6$ jest to $M = 3 \cdot 21 = 63$ bity.

```
def get_all_binary(m, n):
    samples_count = min(n, 2 ** m)

    random_numbers = set()
    while len(random_numbers) < samples_count:
        random_numbers.add(random.getrandbits(m))
    return [format(num, f'0{m}b') for num in random_numbers]

chromosomes = get_all_binary(sum(ms), n)
```

`random.getrandbits(M)` losuje liczbę o M losowych bitach, a zbiór `set` pilnuje, żeby chromosomy się nie powtarzały (losowanie bez zwracania). `format(num, f'0{m}b')` zapisuje liczbę jako ciąg zer i jedynek o stałej długości M, z zerami dopisanymi z przodu, np. dla $M = 8$ liczba 5 to `'00000101'`.

3.4 Krok 4 – podział chromosomu na zmienne

```
def split_chromosome(chromosome, ms):
    result = []
    start = 0
    for m in ms:
        result.append(chromosome[start:start + m])
        start += m
    return result
```

Chromosom jest cięty na n odcinków o długościach $m_1, \ldots, m_n$:

$$\underbrace{b_1 \ldots b_{m_1}}_{x_1} \underbrace{b_{m_1+1} \ldots b_{m_1+m_2}}_{x_2} \cdots \underbrace{\ldots b_M}_{x_n}$$

3.5 Krok 5 – zamiana na liczby dziesiętne

```
def get_decimals(binar):
    result = [int(x, 2) for x in binar]
    return result
```

Każdy odcinek jest zamieniany na liczbę całkowitą, np. `int('10110', 2)` $= 22$.

3.6 Krok 6 – skalowanie do przedziału $[a, b]$

Liczba całkowita $k \in \{0, \ldots, 2^{m_i} - 1\}$ jest odwzorowywana liniowo na przedział zmiennej i zaokrąglana do d_i miejsc:

$$x_i = a_i + \frac{(b_i - a_i)\, k}{2^{m_i} - 1}.$$

```
def get_scale(decimals, data, ms):
    result = [round(var_data['a'] + ((var_data['b'] - var_data['a']) * i
        ) / (2 ** m_val - 1), var_data['d'])
                for i, var_data, m_val in zip(decimals, data, ms)]
    return tuple(result)
```

Przykład dla $d = 1$ ($L = 21$, $m = 5$): $k = 22 \Rightarrow x = -1 + 2 \cdot 22/31 = 0{,}419 \Rightarrow 0{,}4$ po zaokrągleniu.

3.7 Krok 7 – dekodowanie populacji i wybór najlepszego

```
points = []
for chromosome in chromosomes:
    parts = split_chromosome(chromosome, ms)
    to_decimal = get_decimals(parts)
    points.append(get_scale(to_decimal, data, ms))

min_point = points[0]
fun_min = f(*min_point)
for pt in points:
    candidate = f(*pt)
    if candidate < fun_min:
        min_point = pt
        fun_min = candidate
return min_point
```

Każdy chromosom jest dzielony na odcinki, zamieniany na liczby i skalowany, co daje punkt $(x_1, \ldots, x_n)$. Następnie wyszukiwany jest punkt o najmniejszym f. Wartość funkcji dla bieżącego minimum jest zapamiętana w zmiennej `fun_min`.

4 Testy dokładności

Plan badań: $n \in \{3, 5, 7, 9\}$ i $d \in \{1, 2, 3, 6\}$. Każda zmienna ma ten sam przedział $[-1, 1]$ i tę samą dokładność d. Liczebność populacji N dobrano osobno dla każdego d jako część liczby 2^m (liczby różnych ciągów bitów jednej zmiennej): $N = 2^m/16$, $2^m/4$, $2^m/2$ oraz 2^m. Dzięki temu populacja rośnie razem z dokładnością, a porównanie różnych d odbywa się przy tej samej części 2^m. Każdą konfigurację uruchomiono 10 razy z różnymi ziarnami generatora; miarą dokładności jest najlepsza znaleziona wartość f (optimum $= 0$). Pełny kod testów znajduje się w pliku `lab2_testy.py`.

```
for n in [3, 5, 7, 9]:
    for d in [1, 2, 3, 6]:
        m = find_m(2 * 10 ** d + 1)
        for N in [2 ** m // 16, 2 ** m // 4, 2 ** m // 2, 2 ** m]:
            for r in range(10):
                random.seed(seed)
                data = [{'a': -1, 'b': 1, 'd': d} for _ in range(n)]
                p = calc(data, N, func_nd)
                wyniki.append(func_nd(*p))
```

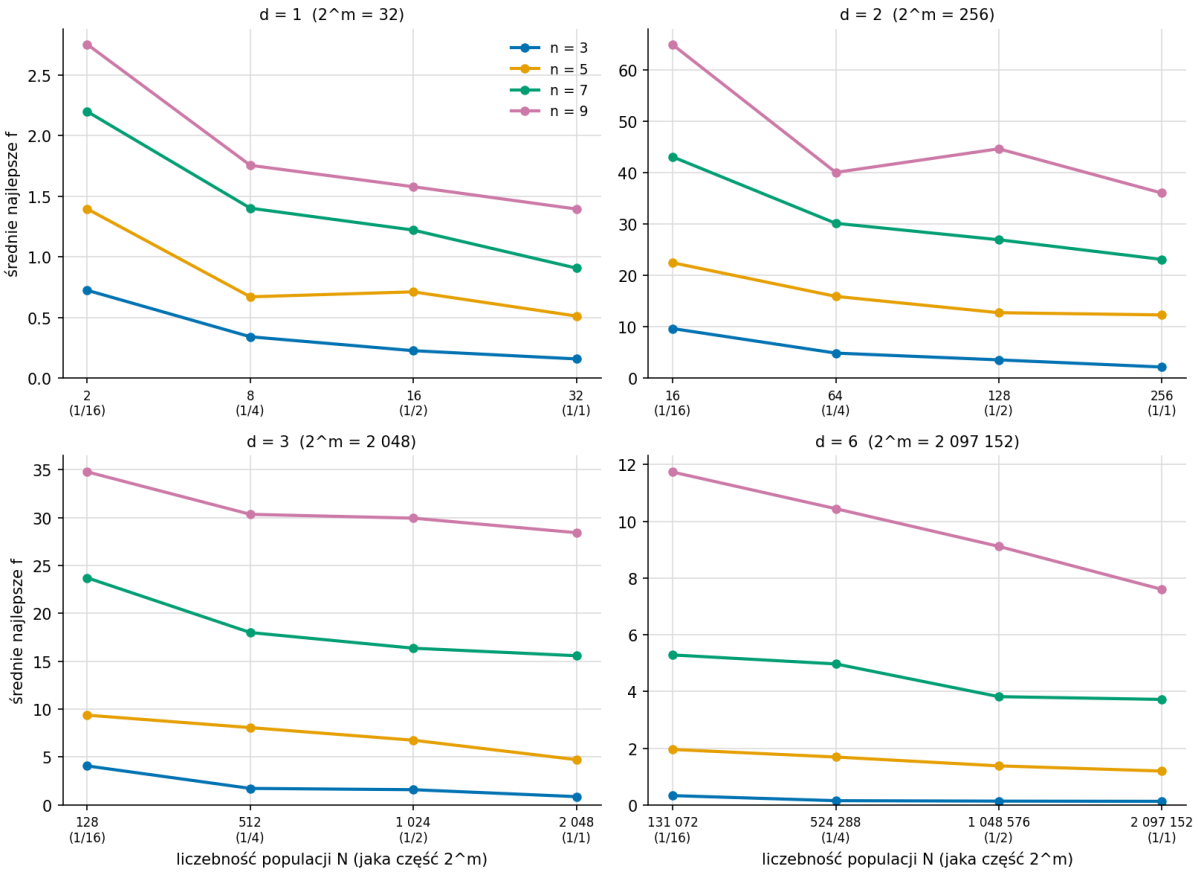
4.1 Parametry kodowania

d	$L = 2 \cdot 10^d + 1$	m [bity]	2^m	badane N
1	21	5	32	2, 8, 16, 32
2	201	8	256	16, 64, 128, 256
3	2 001	11	2 048	128, 512, 1 024, 2 048
6	2 000 001	21	2 097 152	131 072, 524 288, 1 048 576, 2 097 152

Tabela 1: Liczba bitów jednej zmiennej i badane liczebności populacji ($x_i \in [-1, 1]$)

Chromosom osobnika ma $M = n \cdot m$ bitów, np. dla $d = 6$ i $n = 9$ jest to 189 bitów. Ponieważ losowane są całe chromosomy, wartości pojedynczej zmiennej mogą się w populacji powtarzać, nawet gdy $N \leq 2^m$.

4.2 Wpływ liczebności populacji

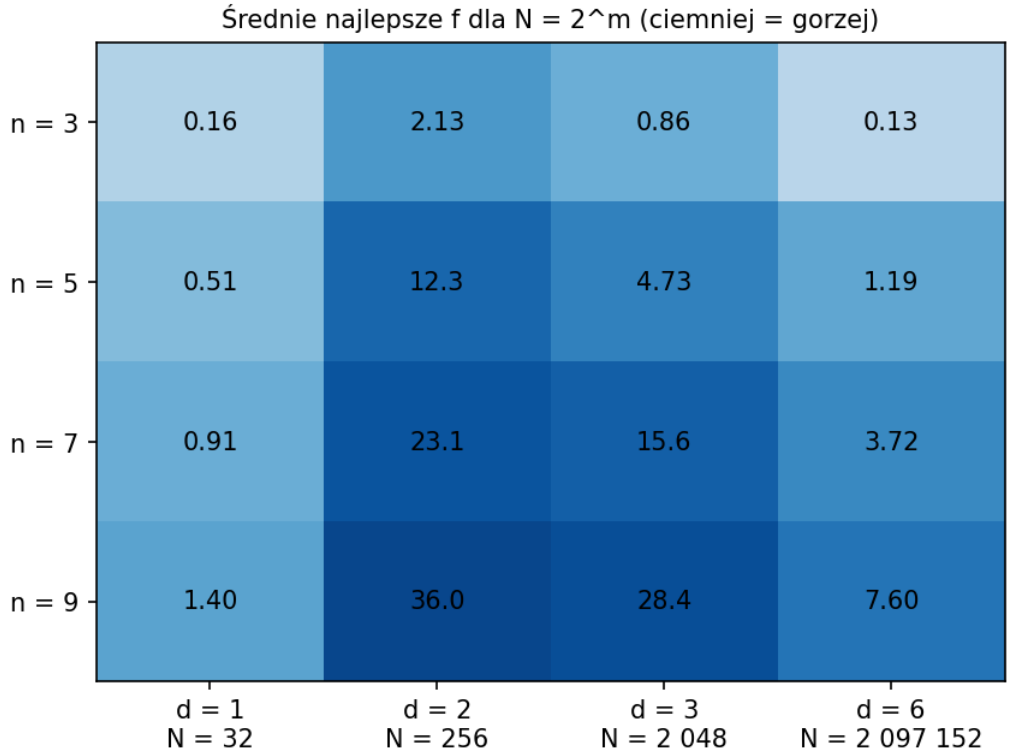


Rysunek 1: Średnia najlepsza wartość f w funkcji N osobno dla każdego d (w nawiasie: jaką częścią 2^m jest N); 10 uruchomień na punkt

Przy tak dobranym N zwiększanie populacji poprawia wynik dla każdego d. Zwykle największy zysk daje przejście z 1/16 do 1/4 części 2^m, a dalsze zwiększanie N pomaga coraz mniej. Przykładowo dla $n = 3$ średnie najlepsze f spada z 0,72 do 0,16 przy $d = 1$, z 9,6 do 2,1 przy $d = 2$ i z 4,1 do 0,86 przy $d = 3$. Dla $d = 6$ spadek jest mniejszy (z 0,33 do 0,13), bo już $N = 131\,072$ to bardzo dużo punktów. Pojedyncze przypadki, w których wynik dla większego N jest nieco gorszy ($n = 5$, $d = 1$ dla $N = 8$ i 16 oraz $n = 9$, $d = 2$ dla $N = 64$ i 128), wynikają z losowości przy 10 powtórzeniach.

Wraz z liczbą zmiennych wynik szybko się pogarsza: przy $d = 6$ i $N = 2\,097\,152$ średnie najlepsze f wynosi 0,13 dla $n = 3$ i 7,6 dla $n = 9$.

4.3 Wpływ dokładności d



Rysunek 2: Średnia najlepsza wartość f dla n zmiennych i dokładności d przy populacji $N = 2^m$

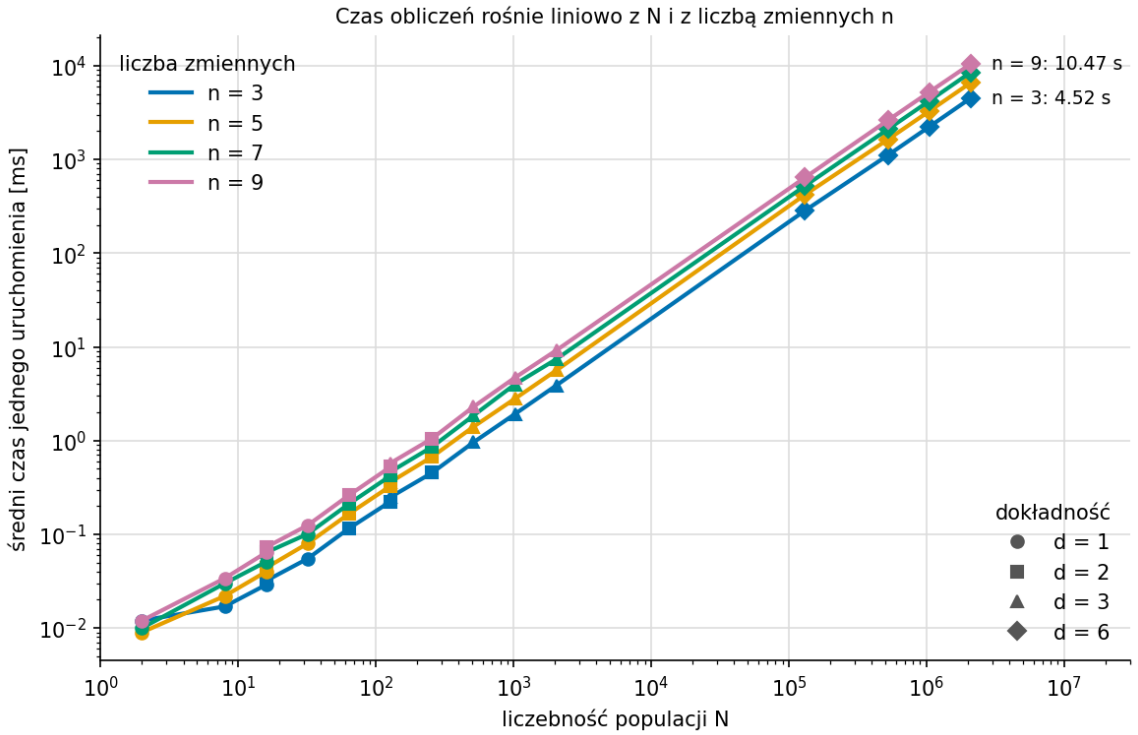
Przy $N = 2^m$ wyniki dla $d = 2$ i $d = 3$ są wyraźnie gorsze niż dla $d = 6$, bo sprawdzanych jest tylko 256 lub 2048 punktów, a dla $d = 6$ ponad 2 miliony. Dla $n \geq 5$ najlepsze wyniki daje $d = 1$, choć sprawdza tylko 32 punkty (dla $n = 3$ jest nieco gorsze od $d = 6$: 0,16 wobec 0,13). Przy $d = 1$ zmienne przyjmują wartości będące wielokrotnościami 0,1, czyli leżą dokładnie w minimach lokalnych funkcji ($\cos(20\pi \cdot 0{,}1k) = 1$), więc $f = \sum x_i^2$.

4.4 Czas obliczeń

Czas jednego wywołania `calc` zmierzono dla tych samych konfiguracji co w testach dokładności: $n \in \{3, 5, 7, 9\}$, $d \in \{1, 2, 3, 6\}$ i $N \in \{2^m/16,\ 2^m/4,\ 2^m/2,\ 2^m\}$. Każdą konfigurację uruchomiono 5 razy po kolei w jednym procesie (bez innych obciążeń) i uśredniono wynik. Pomiar obejmuje całą funkcję `calc`, bez linii `print(points)`, która służy tylko do podglądu populacji i nie jest częścią algorytmu. Środowisko: AMD Ryzen 9 9950X3D, Python 3.13 (plik `lab2_testy_czasowe.py`).

```
for d in [1, 2, 3, 6]:
    m = find_m(2 * 10 ** d + 1)
    for N in [2 ** m // 16, 2 ** m // 4, 2 ** m // 2, 2 ** m]:
        for n in [3, 5, 7, 9]:
            data = [{'a': -1, 'b': 1, 'd': d} for _ in range(n)]
            for r in range(5):
                random.seed(r)
                t = time.perf_counter()
                calc(data, N, func_nd)
```

```
10    czasy.append(time.perf_counter() - t)
```



Rysunek 3: Średni czas jednego uruchomienia `calc` (obie osie logarytmiczne); linia oznacza liczbę zmiennych n, kształt punktu – dokładność d

d	N	liczba zmiennych n			
		3	5	7	9
1	2	0,01 ms	0,01 ms	0,01 ms	0,01 ms
	8	0,02 ms	0,02 ms	0,03 ms	0,03 ms
	16	0,03 ms	0,04 ms	0,05 ms	0,06 ms
	32	0,06 ms	0,08 ms	0,10 ms	0,12 ms
2	16	0,03 ms	0,04 ms	0,06 ms	0,07 ms
	64	0,12 ms	0,17 ms	0,21 ms	0,26 ms
	128	0,22 ms	0,33 ms	0,42 ms	0,53 ms
	256	0,45 ms	0,67 ms	0,86 ms	1,1 ms
3	128	0,25 ms	0,36 ms	0,47 ms	0,58 ms
	512	0,96 ms	1,4 ms	1,9 ms	2,3 ms
	1 024	1,9 ms	2,8 ms	4,0 ms	4,7 ms
	2 048	3,9 ms	5,6 ms	7,4 ms	9,2 ms
6	131 072	284,8 ms	420,6 ms	522,6 ms	641,3 ms
	524 288	1,11 s	1,63 s	2,10 s	2,62 s
	1 048 576	2,24 s	3,28 s	4,21 s	5,24 s
	2 097 152	4,52 s	6,59 s	8,52 s	10,47 s

Tabela 2: Średni czas jednego uruchomienia `calc` (5 powtórzeń)

Czas zależy od liczby sprawdzanych punktów N i od liczby zmiennych n, a sama dokładność d wpływa na niego tylko przez to, jak duże N wybrano. Na wykresie punkty dla różnych d leżą na tej samej prostej. Czas rośnie liniowo z N: dwukrotne zwiększenie populacji wydłuża obliczenia mniej więcej dwukrotnie (dla $d = 6$ i $n = 9$: 2,62 s, 5,24 s i 10,47 s). Rośnie też liniowo z liczbą zmiennych: przy $d = 6$ i $N = 2\,097\,152$ wynosi 4,52 s dla $n = 3$ i 10,47 s dla $n = 9$, czyli ok. 1 s na każdą dodatkową zmienną. Dla $d \leq 3$ populacja liczy co najwyżej 2048 osobników, więc nawet najdłuższy przypadek ($d = 3$, $n = 9$) trwa tylko 9,2 ms.

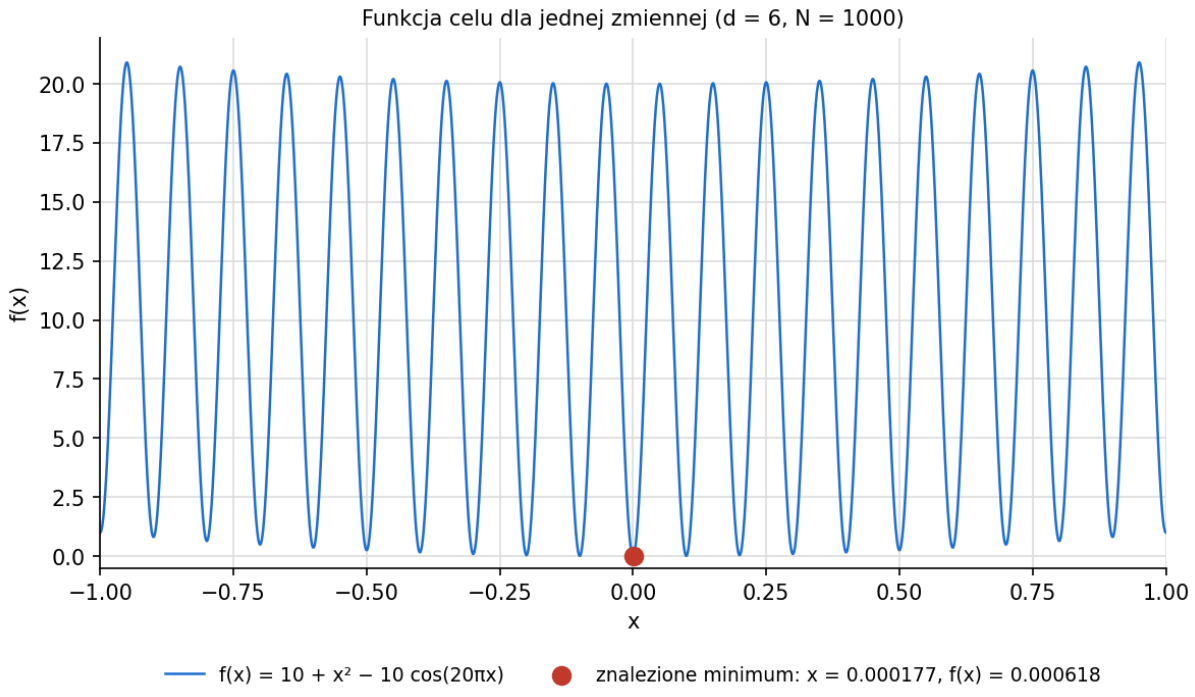
5 Wizualizacja dla jednej zmiennej

Aby wynik dało się pokazać na zwykłym wykresie w osiach x i y, algorytm uruchomiono dla jednej zmiennej ($n = 1$). Do funkcji celu trafia wtedy tylko x, a wykres przedstawia

$$f(x) = 10 + x^2 - 10\cos(20\pi x), \qquad x \in [-1, 1].$$

Dla $d = 6$ mamy $L = 2\,000\,001$ i $m = 21$ bitów, a populacja liczy $N = 1000$ punktów (plik `lab2_wyniki/wykres_1d.py`).

```
random.seed(7)
data = [{'a': -1, 'b': 1, 'd': 6}]     # jedna zmienna x
(x_min,) = calc(data, 1000, func_nd)   # x = 0.000177
f_min = func_nd(x_min)                 # f = 0.000618
```



Rysunek 4: Funkcja celu $f(x)$ dla jednej zmiennej z zaznaczonym znalezionym minimum ($d = 6$, $N = 1000$)

Funkcja ma wiele minimów lokalnych występujących cyklicznie. Najlepszy z 1000 wylosowanych punktów to $x = 0{,}000177$ z $f(x) = 0{,}00062$. Leży on w dolinie minimum

globalnego $x = 0$, $f(0) = 0$, więc w tym uruchomieniu algorytm trafił w okolicę minimum globalnego. Dla jednej zmiennej jest to łatwe, bo dolin jest tylko 21, a 1000 punktów pokrywa je wszystkie.

6 Wnioski

1. Dokładność wyniku szybko spada z liczbą zmiennych. Dla $d = 6$ i $N = 2\,097\,152$ średnie najlepsze f wynosi 0,13 ($n = 3$), 1,19 ($n = 5$), 3,72 ($n = 7$) i 7,60 ($n = 9$).

2. Każdy osobnik to jeden chromosom o długości $m_1 + \cdots + m_n$ bitów, który przy dekodowaniu jest dzielony na odcinki odpowiadające kolejnym zmiennym.

3. Przy N dobranym jako część 2^m zwiększanie populacji poprawia wynik dla każdego d, np. dla $n = 3$ przejście z $N = 2^m/16$ do $N = 2^m$ obniża f z 0,72 do 0,16 ($d = 1$) i z 9,6 do 2,1 ($d = 2$).

4. Czas obliczeń rośnie liniowo z N i z liczbą zmiennych; najdłuższy przypadek ($d = 6$, $n = 9$, $N = 2\,097\,152$) trwa 10,5 s, a dla $d \leq 3$ każde uruchomienie trwa poniżej 10 ms.

5. Dla $n \geq 5$ przy $N = 2^m$ najlepsze wyniki daje $d = 1$ mimo tylko 32 punktów, bo wartości co 0,1 pokrywają się z minimami lokalnymi funkcji.

6. Metoda nie wykorzystuje informacji o dobrych punktach, więc w praktyce trafia w minima lokalne, a nie w minimum globalne; dla $n \geq 5$ i $d \geq 2$ średnie najlepsze f jest większe od 1.